

RecipeBank

Product Requirements Document

Version 1.0 | For AI Implementation Agent

Document Status	Draft – Ready for Engineering
Product	RecipeBank Web Application
Audience	AI Coding Agent / Developers
Prepared For	Engineering Implementation

1. Executive Summary

RecipeBank is a full-stack, multi-user recipe management web application that replaces fragmented note-keeping workflows with a centralized, AI-powered platform. Users can import recipes from URLs (blogs, Pinterest, TikTok), manage a personal recipe library, plan meals on a calendar, build smart shopping lists, and discover recipes through natural-language, semantic search — all from any device.

This document is written for an AI coding agent and provides complete, unambiguous specifications for every feature, page, data model, and interaction state required to implement RecipeBank from scratch.

2. Product Goals & Success Metrics

2.1 Primary Goals

- Replace Apple Notes-style recipe storage with a purpose-built, searchable recipe bank.
- Enable one-click import from any recipe URL, TikTok video, or Pinterest pin.
- Deliver a semantic AI search engine that understands vibes, flavors, cultures, and archetypes — without requiring users to tag recipes manually.
- Provide meal planning, shopping list generation, and cost calculation in a single application.

- Support multiple independent user accounts with password-protected authentication.
- Be fully responsive across desktop and mobile devices.

2.2 Success Metrics

Metric	Target
Recipe import success rate	≥ 95% of valid URLs parse correctly
Search relevance	User accepts top-3 suggestion ≥ 70% of the time
Serving scale accuracy	All unit conversions within ±1% of correct value
Mobile usability	All core flows completable on 375px viewport
Page load (initial)	< 2 seconds on broadband
Authentication security	Bcrypt-hashed passwords, JWT sessions

3. Recommended Tech Stack

The following stack is recommended. The implementing agent may substitute equivalents, but must meet all functional requirements.

Layer	Technology	Notes
Frontend	React + TypeScript	Tailwind CSS for styling; React Query for data fetching
Backend	Node.js + Express or Next.js API Routes	REST or GraphQL
Database	PostgreSQL	Primary relational store
Vector search	pgvector (PostgreSQL ext.)	Semantic recipe search embeddings
AI / Embeddings	OpenAI API (GPT-4o + text-embedding-3-small)	Recipe parsing, metadata generation, search

Layer	Technology	Notes
Web scraping	Playwright + custom parsers	URL & TikTok/Pinterest import
Auth	JWT + bcrypt	Session management
File storage	AWS S3 or Cloudflare R2	Recipe photos
Hosting	Vercel / Railway / Fly.io	Serverless-friendly

4. Core Data Models

The following models represent the primary database schema. All tables should include `created_at` and `updated_at` timestamps unless noted.

4.1 User

Field	Type	Description
id	UUID PK	Primary key
username	VARCHAR(64)	Unique, URL-safe display name
email	VARCHAR(255)	Unique; used for login
password_hash	TEXT	bcrypt hash; never store plaintext
display_name	VARCHAR(128)	Shown in UI
avatar_url	TEXT	Optional profile photo
created_at	TIMESTAMP	Account creation time

4.2 Recipe

Field	Type	Description
id	UUID PK	Primary key
user_id	UUID FK	Owner (references User)
title	TEXT	Recipe name
source_url	TEXT	Original import URL (nullable)
source_type	ENUM	url tiktok pinterest manual
description	TEXT	AI-generated or user-written summary
photo_url	TEXT	Primary header photo (S3/R2 URL)
base_servings	INTEGER	Canonical serving count
current_servings	INTEGER	User-saved scaled serving size
prep_time_minutes	INTEGER	AI-generated or manual
cook_time_minutes	INTEGER	AI-generated or manual
total_time_minutes	INTEGER	Computed: prep + cook
complexity	ENUM	easy medium hard
dish_type	TEXT[]	e.g. [pasta, soup]; AI-generated
cuisine	TEXT[]	e.g. [Italian, Caribbean]; AI-generated
flavor_profile	TEXT[]	e.g. [nutty, spicy]; AI-generated
is_favorite	BOOLEAN	User-starred recipe
cook_count	INTEGER	Number of times logged as cooked
last_cooked_at	TIMESTAMP	Most recent cook log entry
embedding	vector(1536)	pgvector semantic search embedding
raw_import_json	JSONB	Original parsed import data (archived)

4.3 IngredientGroup & Ingredient

Ingredients are organized into named groups within a recipe (e.g., "Chicken", "Sauce", "Other Ingredients"). Each group has an ordered list of ingredient items.

IngredientGroup Field	Type	Description
id	UUID PK	Primary key
recipe_id	UUID FK	Parent recipe
name	TEXT	Group label (e.g. "Sauce"); nullable for default group
sort_order	INTEGER	Display position

Ingredient Field	Type	Description
id	UUID PK	Primary key
group_id	UUID FK	Parent IngredientGroup
name	TEXT	Ingredient name (e.g. "crushed tomatoes")
quantity	DECIMAL	Base quantity at base_servings; NULL when unit is vague (e.g. "to taste")
unit	TEXT	Stored in canonical form (see 7.4). Vague values such as "to taste" or "as needed" stored here with quantity = NULL.
preparation	TEXT	e.g. "finely chopped"; nullable
is_optional	BOOLEAN	Whether ingredient is optional; defaults to false
sort_order	INTEGER	Display position within group

4.4 RecipeStep

Field	Type	Description
id	UUID PK	Primary key
recipe_id	UUID FK	Parent recipe
step_number	INTEGER	1-based display order
content	TEXT	Full step instruction text

Field	Type	Description
section	TEXT	Optional section header (e.g. "Make the sauce")

4.5 RecipeNote

Field	Type	Description
id	UUID PK	Primary key
recipe_id	UUID FK	Parent recipe
user_id	UUID FK	Note author
content	TEXT	Note text
created_at	TIMESTAMP	When the note was written
updated_at	TIMESTAMP	Last edit timestamp

4.6 RecipeEdit

Tracks user-made changes to the original imported recipe so the original and edits remain separately accessible.

Field	Type	Description
id	UUID PK	Primary key
recipe_id	UUID FK	Parent recipe
user_id	UUID FK	Editor
field_name	TEXT	Name of field changed (e.g. ingredient.quantity)
original_value	TEXT	Value before edit
new_value	TEXT	Value after edit
edit_type	ENUM	ingredient step serving unit meta

Field	Type	Description
created_at	TIMESTAMP	When the edit was made

4.7 CookLog

Field	Type	Description
id	UUID PK	Primary key
recipe_id	UUID FK	Recipe that was cooked
user_id	UUID FK	User who cooked it
cooked_at	DATE	Date it was cooked
notes	TEXT	Optional free-text session note

4.8 MealPlan

Field	Type	Description
id	UUID PK	Primary key
user_id	UUID FK	Owner
recipe_id	UUID FK	Planned recipe
planned_date	DATE	Calendar date for this recipe
note	TEXT	Optional note for the day

4.9 ShoppingList & ShoppingItem

ShoppingList Field	Type	Description
id	UUID PK	Primary key
user_id	UUID FK	Owner
recipe_id	UUID FK	Source recipe (nullable for general list)
name	TEXT	List label (auto or user-set)
is_general	BOOLEAN	True for the master general list
created_at	TIMESTAMP	Creation time

ShoppingItem Field	Type	Description
id	UUID PK	Primary key
list_id	UUID FK	Parent ShoppingList
ingredient_id	UUID FK	Source Ingredient (nullable)
name	TEXT	Display name
quantity_needed	DECIMAL	Scaled quantity
unit	TEXT	Unit of measure
is_checked	BOOLEAN	Crossed off in store
store_quantity	DECIMAL	Package size user paid for
store_unit	TEXT	Package unit (oz, g, count, etc.)
store_price	DECIMAL	Price paid for store_quantity
calculated_cost	DECIMAL	Computed: (qty_needed / store_quantity) * store_price

5. Authentication & User Management

5.1 Registration

- Fields: email, username, display name, password (confirmed).
- Password requirements: minimum 8 characters, at least one number or special character.
- Email must be unique across all users.
- Username must be unique, 3–64 characters, alphanumeric with underscores.
- Upon success, user is automatically logged in and redirected to the Home page.

5.2 Login

- Login accepts email OR username + password.
- Sessions managed via signed JWT stored in an httpOnly cookie.
- JWT expiry: 30 days with sliding renewal on activity.
- On failed login: show generic error (do not reveal whether email exists).
- Include a "Remember me" checkbox; if unchecked, session expires on browser close.

5.3 Password Reset

- "Forgot password" triggers an email with a one-time reset link (expires in 1 hour).
- Link leads to a password reset form; on success, all existing sessions are revoked.

5.4 Account Settings

- User may change display name, avatar, email, and password.
- Changing email requires re-verification.
- Account deletion removes all associated recipes, lists, logs, and plans after a 30-day grace period.

6. Page-by-Page Specifications

6.1 Global Navigation

A persistent top navigation bar appears on every page after login.

- Left: RecipeBank logo / wordmark. Clicking returns to Home.
- Center (desktop): Nav links — Home, Recipe Bank, Calendar, Shopping List.
- Right: User avatar / display name with dropdown: Profile, Settings, Log Out.
- Mobile: Hamburger icon opens a slide-out drawer containing all nav links.
- Active page link is visually highlighted.

- The nav bar does not display on the Login or Registration pages.

6.2 Home Page

6.2.1 Hero Section

The page opens with a full-viewport-width hero area displaying:

- Large heading: "Welcome back, [display_name]! What are we cooking today?"
- A contextual sub-heading showing today's date.
- A prominent search bar (same AI search engine as Recipe Bank, Section 6.3).

6.2.2 Suggested Meals Gallery

Below the hero, a horizontally scrollable gallery of recipe cards is displayed. Cards are randomly selected — one per category — from the following required categories:

- Favorites (is_favorite = true)
- Pasta (dish_type contains 'pasta')
- Rice (dish_type contains 'rice')
- Flavor/Culture Spotlight: rotate randomly among Cajun, Caribbean, Indian, Korean, etc.

Selection rules:

- Each card must be from a different recipe — no recipe may appear twice in the gallery, even if it qualifies for multiple categories.
- If a category has no recipes, the slot is omitted (not filled with an incorrect category).
- Randomization is re-seeded on each page load.

Recipe Card (Gallery)

- Photo (full bleed, 16:9 crop). If no photo, show placeholder (dark gray background + centered plate-and-utensils SVG icon in medium gray).
- Recipe title overlaid at bottom-left of photo in white, semi-transparent dark scrim behind text for legibility.
- Category pill badge at top-left (e.g., "Favorites", "Pasta").
- Card is clickable and navigates to the Recipe Page.
- Card min-width: 280px; gap: 16px; horizontal scroll with snap.

6.2.3 Recently Cooked Section

Below the gallery, show a smaller horizontal strip titled "Recently Cooked" — the 5 most recent CookLog entries, deduplicated by recipe. Same card format as the gallery.

6.3 Recipe Bank Page

6.3.1 Layout

The Recipe Bank is a searchable grid of all recipes belonging to the logged-in user.

- Top: AI-powered search bar (full-width on mobile; centered on desktop with max-width 700px).
- Below search: horizontal filter chips (All, Favorites, by Cuisine, by Dish Type, by Complexity). Chips are dynamically generated from the user's actual recipe metadata. Multiple chips can be active simultaneously.
- Below filters: recipe grid (2 columns mobile, 3 columns tablet, 4 columns desktop).
- Top-right: Add Recipe button (primary action, opens import modal).
- Grid supports infinite scroll or paginated navigation (20 recipes per page).

6.3.2 Recipe Grid Card

- Same photo / placeholder behavior as Home page cards.
- Recipe name below photo, bold, 1-line truncated.
- Secondary line: cuisine tag and total time (e.g., "Italian · 45 min").
- Favorite star icon: top-right corner of card; filled when `is_favorite = true`.
- Clicking the card opens a recipe preview drawer (not full navigation) — see Section 6.3.3.

6.3.3 Recipe Preview Drawer

A slide-in panel (right side, desktop) or bottom sheet (mobile) previewing the recipe before the user fully opens it.

- Contains: photo, title, cuisine, dish type, complexity, total time, description, and first 5 ingredients.
- Two buttons: "View Full Recipe" (navigates to Recipe Page) and "Add to Calendar" (opens mini-calendar date picker).
- Drawer is dismissible by clicking outside or pressing Escape.

6.3.4 AI Search Engine

The search bar uses vector similarity search (pgvector) over recipe embeddings. An embedding is generated for each recipe on import (and regenerated if the recipe changes substantially). The embedding is computed over a concatenated string of: title + description + ingredients + dish_type + cuisine + flavor_profile.

Search query handling:

- User input is embedded at query time using the same model.
- Cosine similarity is computed against all recipes for that user.
- Top results are returned ranked by similarity score.
- Results update in real-time as the user types (debounce: 300ms).

The search must handle all of the following query types without manual tags:

- Vibe-based: "romantic date night", "cozy winter meal"
- Ingredient-specific: "sour cream", "canned tomatoes"
- Ingredient-broad: "chicken", "beans"
- Flavor profile: "nutty", "cajun", "tangy"
- Cultural: "Caribbean", "Italian", "Chinese"
- Archetype: "pasta dish", "rice and meat", "stir fry"
- Filters (applied after semantic ranking) narrow results by metadata fields.

6.3.5 Add Recipe Modal (Import)

Triggered by the "Add Recipe" button. A modal dialog with two tabs:

Tab 1: Import from URL / Video

- Single text input for a URL.
- Supported source types: recipe blogs (any), Pinterest pin, TikTok video URL.
- On submit, backend scrapes / processes the URL and extracts: title, ingredients (with groups if present), steps, photo, and any existing metadata.
- For TikTok: transcribe audio or parse caption/comments using AI to extract recipe. If photo unavailable, show video thumbnail.
- For Pinterest: follow the pin's linked URL if it leads to a recipe page; otherwise parse the pin description.
- After parsing, show a preview of the extracted data with an option for the user to confirm or edit before saving.
- AI generates and auto-fills: complexity, prep_time, cook_time, dish_type, cuisine, flavor_profile, and description.
- User may override any AI-generated field before saving.

Tab 2: Manual Entry

- Form with fields: Title, Photo upload, Description, Source URL (optional), Servings, Prep time, Cook time, Ingredients (grouped), Steps.
- AI-generation button: after title and ingredients are entered, user can click "Auto-fill AI fields" to generate complexity, dish_type, cuisine, flavor_profile, and description.
- Ingredient entry supports adding groups and reordering lines via drag-and-drop.

6.4 Recipe Page

Each recipe has a dedicated full-page route (e.g., /recipes/[id]). The page is structured top-to-bottom as described below.

6.4.1 Header / Hero

- Full-width photo at top. If no photo, use the plate-and-utensils placeholder.
- Recipe title overlaid in the bottom-left corner of the image, always visible regardless of photo content. Use a semi-transparent dark gradient scrim (linear-gradient from bottom) behind the title. Title text: white, bold, large.
- Favorite star icon in the top-right of the image.
- Title remains visible (sticky or scroll-locked) as the user scrolls. On mobile, the photo collapses to a compact header bar with the title visible.

6.4.2 Source & Metadata Bar

- Source URL displayed as a clickable link with the domain name: "Source: [domain.com]" (e.g., "Source: seriouseats.com").
- Metadata pills: Complexity · Prep · Cook · Total Time · Dish Type · Cuisine.
- These are editable in Edit Mode.

6.4.3 Mode Toggle

A segmented control switches between four modes. The control should be sticky (always visible while scrolling the recipe):

Mode	Behavior
View (default)	Read-only display. Ingredients as bullet list; steps as numbered list.
Edit	All fields become editable in-line. Confirms on Save; Cancel reverts.
List	Checkbox appears next to each ingredient. Checked items can be added to Shopping List.
Shopping	Extends List mode with price/quantity input per checked item. Calculates cost.

6.4.4 Servings Scaler

Displayed prominently near the top of the ingredient section:

- Editable integer input for serving count. Only positive integers accepted; non-integer or non-positive input is rejected with inline validation.
- On change: all ingredient quantities update in real-time using the ratio ($\text{new_servings} / \text{base_servings}$). Quantities round to 2 decimal places; display fractions where natural (e.g., $0.5 \rightarrow 1/2$).
- A "Save as new default" button appears after scaling, which updates `current_servings` and recalculates all stored ingredient quantities.

6.4.5 Ingredients Section

Ingredients are displayed in their groups (see IngredientGroup model). Each group has a bold section label. Items within a group display as: [quantity] [unit] [name], [preparation].

Special display cases:

- Vague / unitless ingredients (e.g. "to taste", "as needed"): display as "[name], [unit]" with no quantity. Example: "salt, to taste".
- Optional ingredients: rendered in semi-transparent gray text with "(optional)" appended in parentheses after the ingredient name. Example: "1/2 tsp parsley flakes (optional)". The optional label uses a muted color (approximately 50% opacity gray) to distinguish from required items without cluttering the list.
- Quantity display: all values < 1 are displayed as proper fractions (e.g. $0.5 \rightarrow 1/2$, $0.25 \rightarrow 1/4$, $0.333... \rightarrow 1/3$, $0.75 \rightarrow 3/4$). Values ≥ 1 with a fractional part display as a mixed number (e.g. $1.5 \rightarrow 1 \frac{1}{2}$). The underlying database value remains a DECIMAL; conversion to fractions is a display-only operation. Use a GCD-based simplification algorithm supporting denominators up to 16.

In Edit Mode:

- Each ingredient line is editable (quantity, unit, name, preparation are separate inline fields).
- Drag handles allow reordering items within and across groups.
- "Add ingredient" and "Add group" buttons appear at the bottom of each group and the overall section.
- Clicking any measurement (quantity + unit) opens a unit conversion popover: user selects target unit, conversion is previewed, and Save applies it. Supported conversions: volume (tsp, tbsp, cup, fl oz, ml, L), weight (g, kg, oz, lb).

6.4.6 Steps Section

Steps display as a numbered list with optional section headers (e.g., "Make the Sauce").

- In Edit Mode, steps are editable text areas. Steps can be reordered by drag-and-drop.
- Section headers can be added, renamed, or deleted.

6.4.7 Notes Section

Below steps, a notes section shows all notes in reverse chronological order.

- Each note shows: note text, timestamp ("Added [Month DD, YYYY]"), and a small edit/delete icon.
- A text area with an "Add Note" button allows new note submission.
- Notes are user-specific (each user sees only their own notes).

6.4.8 Recipe Edits Section

A collapsed accordion at the bottom of the page. Default state: collapsed. Expanded state shows a table of all RecipeEdit records for this recipe.

Field Changed	Original Value	New Value	Date
(example) ingredient.quantity	14 oz	7 oz	Jun 1, 2025

6.4.9 Cook Log Button

- A floating action button (bottom-right, desktop) or sticky bottom bar (mobile) labeled "Log as Cooked Today".
- On click: creates a CookLog entry with today's date. Increments cook_count on the Recipe. Updates last_cooked_at.
- If already logged today: button reads "Cooked Today ✓" and is disabled.
- A secondary link "View cook history" opens a modal listing all CookLog entries for this recipe with dates and any notes.

6.4.10 List Mode

In List mode, each ingredient row gains a checkbox to its left. The user selects which ingredients they need to purchase.

- A floating panel at the bottom of the screen shows: "X items selected — Add to Shopping List."
- Clicking "Add to Shopping List" creates a new ShoppingList linked to this recipe and adds all selected Ingredients as ShoppingItems, using the currently displayed (scaled) quantities.
- This list is also merged into the user's general shopping list.

6.4.11 Shopping Mode

Shopping Mode builds on List Mode. Once items are added to the shopping list, the user can expand each item to input store pricing.

- For each checked item, an inline form appears: [Store Qty] [Store Unit] for \$[Price].
- On input, `calculated_cost` is computed and shown: "Used: [qty] [unit] = \$[cost]".
- Fields are optional; if left blank, cost is shown as "—".
- Total recipe cost (sum of all `calculated_costs`) is displayed at the top of the section.

Shopping Mode Calculation Example

Recipe requires 14 oz of crushed tomatoes.

- User inputs: store package = 28 oz for \$8.99.
- System calculates: $(14 / 28) \times \$8.99 = \4.50 (rounded to nearest cent).
- Displayed as: "Crushed Tomatoes: \$4.50"

6.5 Calendar Page

6.5.1 Layout

- Standard calendar grid, week starting Sunday, ending Saturday.
- Three view modes toggle via a segmented control at the top: Day | Week | Month.
- Previous/Next navigation arrows move forward or back by the current view's unit.
- "Today" button snaps back to the current date.
- Calendar does NOT divide days into meal slots or hour slots — a day is a flat container of recipe cards.

6.5.2 Adding Recipes to Calendar

- Each day cell has an "+" button. Clicking opens a search-and-select modal to pick a recipe.
- Selected recipe appears as a compact card within the day cell: photo thumbnail + recipe name.
- Multiple recipes can be added to one day.
- Drag-and-drop between day cells to move a MealPlan entry.
- Clicking a recipe card in the calendar opens the Recipe Preview Drawer.
- Hovering over a recipe card shows a remove (×) icon.

6.5.3 View: Month

- Standard month grid. Each day shows up to 2 recipe pill chips; overflow shows "+N more" link.
- Clicking "+N more" expands the day cell in a popover.

6.5.4 View: Week

- 7-column layout showing full day cells. All recipes for the week are visible.
- Each day column is tall enough to show all planned recipes without clipping.

6.5.5 View: Day

- Single-day focused view. All planned recipes for that day appear as larger cards with photo, title, cuisine, and total time.
- "Generate Shopping List for Today" button creates a combined ShoppingList from all recipes on the selected day.

6.6 Shopping List Page

6.6.1 Page Structure

The page is organized as two tabs displayed at the top of the page on all screen sizes (both desktop and mobile). There is no side-by-side split pane layout.

- Tab 1 — General: A flat, unified list of all shopping ingredients across all recipe lists, deduplicated.
- Tab 2 — By Recipe: All shopping ingredients listed together but grouped by recipe, with each recipe's name as a bold section header. A filter control at the top of this tab allows the user to narrow the view to a single recipe.

6.6.2 General Tab

- A searchable list of all ShoppingItems across all lists, deduplicated by normalized ingredient name + canonical unit (see deduplication rules below).
- Items are sorted alphabetically by default; user may also sort by recipe source.
- Search bar filters visible items in real-time.
- Each item row: [checkbox] [quantity + unit] [ingredient name] [source recipe(s) badge].
- Checking an item marks it as `is_checked = true`. Checked items move to a "Got It" section at the bottom, visually de-emphasized (strikethrough, reduced opacity).
- A "Clear Checked" button removes all checked items from the list.

Deduplication rules for the General tab:

- Matching is case-insensitive. "salt" and "Salt" are treated as the same ingredient and merged into one row with quantities summed.
- Matching must be conservative to avoid incorrectly merging different ingredients. Only exact name matches (after lowercasing and trimming whitespace) are deduplicated. Partial or substring matches are never automatically merged. Examples of ingredients that must NOT be merged: "garlic cloves" and "garlic powder"; "chicken stock" and "chicken stock concentrate"; "butter" and "peanut butter".
- When quantities are merged, units must be compatible and identical (or normalized equivalents per Section 7.4). Ingredients with incompatible units are displayed as separate rows even if the name matches.
- The source recipe(s) badge lists all recipes contributing to a merged row.

6.6.3 Price Input Interaction

When an item is checked, a non-disruptive inline row expands below it (not a modal, not a page change):

- [Store Qty] [Unit] for \$[Price] — with a "Save" micro-button.
- This row is dismissible by clicking away, pressing Escape, or tapping a small "skip" link.
- Saved price data is stored in ShoppingItem and propagates to Shopping Mode on the Recipe Page.
- If the user skips, the item remains checked but without price data.

6.6.4 By Recipe Tab

The second tab lists all shopping ingredients grouped by their source recipe. Each recipe forms a collapsible section with:

- Recipe name as a bold section header, with a "View Recipe" link inline.
- All ingredient rows for that recipe, with the same checkbox/price interaction as the General tab.
- A "Delete List" button per recipe section (removes that recipe's items from the shopping list; does not affect the recipe itself).

A filter dropdown at the top of the tab allows the user to select a specific recipe and show only that recipe's section, hiding all others. Selecting "All Recipes" (default) shows all sections.

6.6.5 Adding Items Manually

- A text input at the top of the general list allows freeform item entry: type name + optional quantity/unit, press Enter to add.
- Manually added items have no ingredient_id (source = null).

7. AI Features & Import Pipeline

7.1 URL Import Pipeline

When a URL is submitted for import, the following steps execute in sequence on the backend:

1. Detect source type: determine if URL is a TikTok video, Pinterest pin, or generic web URL.
2. Fetch & render: use Playwright (headless Chromium) to fetch the page, handling JavaScript-rendered content.
3. Extract structured data: check for JSON-LD (schema.org/Recipe) in the page head — if present, parse directly. If not, use AI (GPT-4o) to extract recipe data from the rendered HTML.
4. For TikTok: download video metadata and caption. Use AI to transcribe audio or parse caption for recipe steps and ingredients.
5. For Pinterest: follow the pin's linked source URL; parse that page. If the pin has no link, parse the pin's description only.
6. Normalize data: unify extracted ingredients into the Ingredient schema (name, quantity, unit, preparation, group). All unit strings are normalized to their canonical form at this step using the alias mapping in Section 7.4. Vague unit phrases ("to taste", "as needed", etc.) are detected and stored with quantity = NULL.
7. Generate AI metadata: send a structured prompt to GPT-4o requesting: complexity, dish_type, cuisine, flavor_profile, description, prep_time, cook_time.
8. Generate embedding: concatenate all text fields, call text-embedding-3-small, store vector in recipe.embedding.
9. Save to database: persist Recipe, IngredientGroup, Ingredient, and RecipeStep records.
10. Return structured preview to frontend for user confirmation.

7.2 AI Metadata Generation Prompt

The following fields must be AI-generated for every recipe (import or manual after clicking "Auto-fill AI fields"). The backend sends a structured prompt to GPT-4o and parses a JSON response:

Required AI Output Fields

complexity — one of: easy | medium | hard

prep_time_minutes — estimated prep time in minutes (integer)

cook_time_minutes — estimated cook time in minutes (integer)

dish_type — array of strings, e.g. ["pasta", "baked"]

cuisine — array of strings, e.g. ["Italian", "Mediterranean"]

flavor_profile — array of strings, e.g. ["savory", "herbaceous", "tangy"]

description — 2-3 sentence summary of the dish for search and display

All AI-generated fields are pre-filled in the UI but are fully editable by the user before saving. Any field the user overrides is treated as canonical; the AI value is logged but not re-applied unless the user requests regeneration.

7.3 Serving Scaler Logic

All ingredient quantity scaling must use the following formula:

$$\text{scaled_quantity} = (\text{ingredient.quantity} / \text{recipe.base_servings}) \times \text{new_servings}$$

- Only ingredient quantities are scaled — not steps, descriptions, or metadata.
- The scalar is applied to the base ingredient quantities stored in the database (not to previously scaled values) to avoid floating-point drift.

- Ingredients with vague units (quantity = NULL, e.g. "to taste") are never scaled and always display as-is.
- Fraction display rules (display-only; underlying value always stored as DECIMAL): quantities < 1 are always shown as a proper fraction (e.g. 0.5 → 1/2, 0.25 → 1/4, 0.333 → 1/3, 0.75 → 3/4). Quantities ≥ 1 with a fractional part display as a mixed number (e.g. 1.5 → 1 1/2, 2.25 → 2 1/4). Whole-number quantities display as integers. Use a GCD-based simplification algorithm with denominator support up to 16; if a value cannot be expressed as a clean fraction within this constraint, round to the nearest 1/16 and display that fraction.
- Accepted inputs: positive integers only. Decimals and zero are rejected with an inline error message.

7.4 Unit Normalization & Conversion

All unit values are normalized to a canonical form at import time and whenever an ingredient is saved. Normalization ensures that variant spellings and abbreviations are treated as identical throughout the application.

Unit normalization rules:

- Normalization is case-insensitive and strips leading/trailing whitespace.
- All recognized aliases are mapped to a single canonical string before storage. The table below lists required aliases; the canonical form is what gets stored in the database.

Canonical Unit	Recognized Aliases (stored as canonical)
tsp	tsp, teaspoon, teaspoons, t
tbsp	tbsp, tablespoon, tablespoons, tbs, T, Tbsp
cup	cup, cups, c
fl oz	fl oz, fluid ounce, fluid ounces, floz
ml	ml, mL, milliliter, milliliters, millilitre
L	L, l, liter, liters, litre, litres
oz	oz, ounce, ounces
lb	lb, lbs, pound, pounds
g	g, gram, grams
kg	kg, kilogram, kilograms

Any unit string not matched to a known alias is stored as-is (lowercase, trimmed) without

conversion. Vague unit strings such as "to taste", "as needed", "a pinch", or "to preference" are stored verbatim in the unit field with quantity set to NULL and are never included in conversion or scaling logic.

Unit conversion (when user selects a target unit in Edit Mode):

- System presents compatible target units based on dimension (volume or weight). Units from different dimensions (e.g. oz to ml) are not offered as conversion options.
- Conversion uses standard factors (e.g., 1 cup = 16 tbsp = 48 tsp = 8 fl oz = 236.6 ml).
- Converted quantity is shown as a preview before saving. The preview uses the fraction display rules from Section 7.3.
- Saving a unit conversion creates a RecipeEdit record.

Dimension	Supported Units
Volume	tsp, tbsp, cup, fl oz, ml, L
Weight	g, kg, oz, lb
Count	No conversion (e.g., 2 eggs)
Vague	No conversion (to taste, as needed, etc.)

7.5 Home Page Suggestion Algorithm

The home page gallery slot selection uses the following logic, executed server-side at page load:

11. Query all recipes for the user.
12. For each required category, build a candidate pool: filter recipes matching category criteria.
13. Shuffle each pool randomly (Fisher-Yates).
14. Pick the first candidate from each pool not already selected (de-duplication check across all selected recipes).
15. If a category pool is empty, that slot is omitted from the gallery.
16. Assemble the final ordered gallery array and return to the client.

8. Responsive Design Requirements

8.1 Breakpoints

Label	Min Width	Key Layout Changes
Mobile	0px	Single-column layout; hamburger nav; bottom-sheet drawers; tabbed navigation.
Tablet	768px	2-column recipe grid; slide-in drawers; expanded tab content areas.
Desktop	1280px	4-column recipe grid; full nav bar; floating panels; wider content containers.

8.2 Mobile-Specific Requirements

- Recipe page: photo collapses to a compact sticky header as user scrolls. Title remains visible.
- Mode toggle (View/Edit/List/Shopping): renders as a scrollable horizontal strip or dropdown selector.
- Ingredient drag-and-drop on mobile: uses long-press to initiate drag.
- Calendar week view: horizontally scrollable on mobile; single visible column at a time with swipe navigation.
- Shopping List: uses the same tabbed interface (General / By Recipe) on all screen sizes.
- All tap targets minimum 44×44px.
- Font size minimum 16px for form inputs to prevent iOS auto-zoom.

9. Edge Cases & Error Handling

9.1 Recipe Import Failures

Scenario	Behavior
URL is not a recipe page	AI extraction fails gracefully; user shown message and offered Manual Entry fallback.
TikTok video has no recipe	Inform user; do not save a partial recipe.
Pinterest pin has no link	Parse pin description only; flag to user that data may be incomplete.
AI metadata generation fails	Save recipe without AI metadata; display a "Generate AI fields" button on the recipe page.

Scenario	Behavior
Duplicate URL detected	Warn user that this URL was already imported; offer to view existing recipe or import again.
Network timeout	Retry up to 3 times with exponential backoff; then surface friendly error.

9.2 Serving Scaler

- Input of 0: show error "Servings must be at least 1".
- Decimal input: show error "Please enter a whole number".
- Very large servings (> 999): show warning "Unusually large serving count — are you sure?" but allow submission.

9.3 Photo Handling

- Missing photo: always show the plate-and-utensils placeholder. Never show a broken image tag.
- User-uploaded photo: validate file type (jpg, png, webp, heic), max size 20MB, auto-convert HEIC to JPEG on server.
- Imported photo URL that becomes unavailable: cache the image to S3/R2 at import time to prevent link rot.

9.4 Empty States

- Recipe Bank with no recipes: show illustration + "Add your first recipe" CTA.
- Search with no results: show "No recipes found for '[query]'" with a suggestion to add a recipe.
- Calendar with no planned meals: show "Nothing planned — add a recipe to get started" in each empty day cell.
- Shopping list with no items: show "Your list is empty — select ingredients from a recipe to get started."
- Home page category with no qualifying recipes: omit that card slot silently.

10. Security & Privacy Requirements

- All API endpoints require authentication. Unauthenticated requests return 401.
- All recipe, note, shopping list, and calendar data is scoped to the authenticated user. Accessing another user's data returns 403.
- Passwords hashed with bcrypt (cost factor ≥ 12).
- JWT signed with a strong secret (HS256 or RS256). Secret stored in environment variable, not in code.
- HTTPS enforced in production. HTTP redirects to HTTPS.
- All user inputs are sanitized to prevent XSS. React's default escaping is used; avoid dangerouslySetInnerHTML.
- SQL queries use parameterized statements to prevent SQL injection.
- Uploaded files are stored in object storage (not served from the application server). Content-type validated on upload.
- Rate limiting on authentication endpoints: max 10 login attempts per IP per 15 minutes.
- All API keys stored in environment variables. Never exposed to the client.

11. API Endpoint Reference

All endpoints are prefixed with `/api/v1`. All requests require `Authorization: Bearer [jwt]` unless noted as public.

11.1 Auth Endpoints (Public)

Method	Path	Description
POST	/auth/register	Register new user
POST	/auth/login	Login; returns JWT
POST	/auth/logout	Invalidate session
POST	/auth/forgot-password	Send password reset email
POST	/auth/reset-password	Apply password reset token

11.2 Recipe Endpoints

Method	Path	Description
GET	/recipes	List all recipes (paginated, with filters)
POST	/recipes	Create recipe (manual)
POST	/recipes/import	Import recipe from URL
GET	/recipes/search	Semantic search (query param: q)
GET	/recipes/suggestions	Home page suggestion slots
GET	/recipes/:id	Get single recipe (full detail)
PATCH	/recipes/:id	Update recipe fields
DELETE	/recipes/:id	Delete recipe
POST	/recipes/:id/favorite	Toggle favorite status
POST	/recipes/:id/cook-log	Log a cook event
GET	/recipes/:id/cook-log	Get cook history for recipe
GET	/recipes/:id/notes	Get notes for recipe
POST	/recipes/:id/notes	Add note to recipe
PATCH	/recipes/:id/notes/:nid	Edit a note
DELETE	/recipes/:id/notes/:nid	Delete a note
GET	/recipes/:id/edits	Get edit history
POST	/recipes/:id/ai-metadata	Re-generate AI metadata

11.3 Calendar Endpoints

Method	Path	Description
GET	/meal-plan	Get meal plans (query: start_date, end_date)
POST	/meal-plan	Add recipe to a date
PATCH	/meal-plan/:id	Move to different date
DELETE	/meal-plan/:id	Remove from calendar

11.4 Shopping List Endpoints

Method	Path	Description
GET	/shopping-lists	Get all lists for user
POST	/shopping-lists	Create a shopping list
GET	/shopping-lists/:id/items	Get items in a list
POST	/shopping-lists/:id/items	Add item to list
PATCH	/shopping-lists/:id/items/:iid	Update item (check, price, qty)
DELETE	/shopping-lists/:id/items/:iid	Remove item from list
POST	/shopping-lists/:id/from-recipe/:rid	Bulk-add ingredients from recipe
DELETE	/shopping-lists/:id/checked	Clear all checked items

12. Non-Functional Requirements

12.1 Performance

- Initial page load (LCP): < 2.5 seconds on a 50 Mbps connection.
- Recipe import pipeline: complete within 30 seconds for standard URLs; surface progress indicator.
- Search results: return within 500ms for queries against libraries up to 500 recipes.
- Serving scale re-calculation: < 50ms (client-side, no network round-trip needed).

12.2 Light & Dark Mode

- The application supports both a light theme (default) and a dark theme. Light mode is the default on first load.
- A theme toggle control is accessible from the user account dropdown (top-right navigation) on all pages.
- The user's theme preference is persisted in their account settings (stored server-side) so it is restored across sessions and devices.
- If no preference is stored, the application defaults to light mode regardless of the OS-level preference.

- Theme is implemented using CSS custom properties (variables). All colors reference variables; no hardcoded color values should appear in component styles.
- All placeholder graphics (plate-and-utensils SVG) must be legible in both themes.
- All color contrast requirements from Section 12.3 apply independently to both the light and dark themes.

12.3 Accessibility

- WCAG 2.1 AA compliance.
- All interactive elements keyboard-navigable.
- All images have descriptive alt text. Placeholder images have alt="Recipe photo placeholder".
- Color contrast ratio $\geq 4.5:1$ for normal text, in both light and dark themes.
- Forms have associated label elements.
- Modal dialogs trap focus and restore focus on close.

12.4 Internationalization

- All displayed text stored in i18n-friendly strings (not hardcoded). Default language: English (US).
- Date formats follow the user's browser locale.
- Number formatting (decimals) follows user locale.

12.5 Browser Support

- Chrome ≥ 110 , Firefox ≥ 110 , Safari ≥ 16 , Edge ≥ 110 .
- iOS Safari ≥ 16 , Chrome for Android ≥ 110 .
- No IE11 support required.

13. Recommended Implementation Order

The following phased order minimizes blocking dependencies and allows incremental testing:

Phase 1 — Foundation

- 17.Database schema and migrations (all models in Section 4).
- 18.Authentication system (Section 5): register, login, JWT, password reset.
- 19.Basic CRUD for recipes: create (manual), read, update, delete.
- 20.Ingredient group + ingredient CRUD with sort ordering.
- 21.Recipe steps CRUD.

Phase 2 — Core Recipe Features

- 22.Recipe page UI: View mode, serving scaler, unit conversion.
- 23.Recipe Bank page: grid, filter chips, placeholder card.
- 24.Notes system.
- 25.Cook log.
- 26.Favorite toggle.
- 27.Recipe Edits tracking system.

Phase 3 — AI & Import

- 28.URL import pipeline (scraping + JSON-LD + AI extraction).
- 29.TikTok and Pinterest handlers.
- 30.AI metadata generation.
- 31.pgvector embedding storage and semantic search.
- 32.Edit Mode with drag-and-drop reordering.

Phase 4 — Planning & Shopping

- 33.Calendar page (Month, Week, Day views).
- 34.Meal Plan CRUD and drag-and-drop.
- 35.Shopping List page with general and per-recipe lists.
- 36.Shopping Mode on Recipe Page with cost calculation.

Phase 5 — Home & Polish

- 37.Home page hero, suggestion algorithm, gallery.
- 38.Recipe Preview Drawer.
- 39.Light/Dark mode theming with CSS custom properties and user preference persistence.
- 40.Responsive design audit across all breakpoints.
- 41.Accessibility audit (WCAG 2.1 AA) in both light and dark themes.
- 42.Performance optimization (image lazy loading, search debounce, caching).
- 43.End-to-end testing of all major flows.

14. Open Questions & Decisions for Product Owner

#	Question	Impact if Unresolved	Answer
1	Should users be able to share recipes with other RecipeBank users, or is the app strictly private per account?	Affects data model (shared recipes, permissions).	No, the app is private per account. There will realistically only be a few users.
2	Is there a maximum number of recipes per user account?	Affects storage budgeting and pgvector index sizing.	No, the app has unlimited recipe storage but there will not be many users or over 500 recipes.
3	Should the app support multiple languages / locales beyond English?	Affects AI prompt design and UI string management.	No, English is sufficient.
4	What TikTok API or scraping strategy is acceptable? (TikTok's ToS restricts scraping.)	May limit TikTok import feature; requires legal review.	Unknown
5	Should there be a freemium tier (e.g., recipe limit) or is RecipeBank fully free to all registered users?	Affects auth model and business logic.	No, RecipeBank is fully free to all.
6	Are admin/moderation tools needed (e.g., to manage accounts)?	Affects user management backend.	There will be a few users at most, so considering this, the decision is up to the AI agent.
7	Should cook logs be shareable or social (e.g., "friends' activity")?	Significant feature scope expansion.	No, they should be functional and private. They should be linked to the calendar plan, but only when the date is past.
8	Should the calendar support recurring meal plans?	Requires recurrence rule logic (RFC 5545).	No.

End of Document — RecipeBank PRD v1.0